

Ridge Regression Homework

Ridge Regression Homework

Question 2 — Ridge with nearly collinear predictors

Setup diagnostics

- $\text{corr}(X_1, X_2) = \mathbf{0.9983}$
- largest singular value of X , $d_{\max} = \mathbf{14.173}$
- smallest singular value of X , $d_{\min} = \mathbf{0.406}$

Why $\beta_1 - \beta_2$ should be far more variable than $\beta_1 + \beta_2$. X_1 and X_2 are both built from the same shared factor L_i plus a small independent perturbation ($0.04 \cdot U_{i1}$, $0.04 \cdot U_{i2}$). So, up to standardization,

- $X_1 + X_2 \approx 2L$ (a strong, well-determined direction — the two columns reinforce each other here), and
- $X_1 - X_2 \approx 0.04(U_1 - U_2)$ (a very weak direction — the two columns nearly cancel).

This is exactly the near-collinearity SVD picks up: X has one large singular value associated with directions where X_1 and X_2 move together, and one very small singular value associated with the direction where they move apart. For (unpenalized) least squares, the variance of an estimated contrast in a given singular direction scales like $1/d_j^2$. Since $d_{\min} \ll d_{\max}$ here (0.406 vs. 14.173, a ratio of about 35), $\text{Var}(\beta_1 - \beta_2) \propto 1/d_{\min}^2$ is enormous relative to $\text{Var}(\beta_1 + \beta_2) \propto 1/d_{\max}^2$ — the simulation below shows this ratio of standard deviations landing almost exactly at that same ~ 35 x. Intuitively: the data barely constrain the *difference* between the two nearly-identical columns, so any small amount of noise gets hugely amplified when we try to separate their individual effects — but the *sum* of their effects is pinned down tightly because that's the direction the data actually vary in.

Simulation results (200 repetitions, seed 43231)

λ	mean β_1	sd β_1	mean β_2	sd β_2	mean $(\beta_1 + \beta_2)$	sd $(\beta_1 + \beta_2)$	mean $(\beta_1 - \beta_2)$	sd $(\beta_1 - \beta_2)$	mean train MSE	mean test MSE
0	1.2901	1.7951	1.7014	1.7979	2.9915	0.1049	-0.4113	3.5915	0.9493	1.0384
0.02	1.4645	0.1447	1.4968	0.1479	2.9613	0.1039	-0.0323	0.2735	0.9597	1.0292
0.2	1.3553	0.0494	1.3593	0.0504	2.7146	0.0954	-0.0041	0.0293	1.0639	1.1253

Interpretation (bias/variance). At $\lambda = 0$ (OLS), β_1 and β_2 individually average out close to their true values but have huge standard deviations (1.80 each), and in particular $\text{sd}(\beta_1 - \beta_2) = 3.59$ dwarfs $\text{sd}(\beta_1 + \beta_2) = 0.10$ — a ~ 34 x gap, matching the $d_{\max}/d_{\min} \approx 35$ ratio almost exactly, as predicted by the singular-value argument above. As λ increases from 0 to 0.02 to 0.2, the individual coefficients and especially the contrast $\beta_1 - \beta_2$ shrink sharply in variance ($\text{sd}(\beta_1 - \beta_2)$ drops from 3.59 $\rightarrow$ 0.27 $\rightarrow$ 0.03), at the cost of some bias creeping into $\beta_1 + \beta_2$ (its mean drifts from 2.99, the true value, down to 2.71 at $\lambda=0.2$) and into the individual slopes (true value 1.5, but mean drifts to 1.36/1.36 at $\lambda=0.2$). Despite that huge variance reduction in the coefficients, mean test MSE barely moves — it's 1.0384 at $\lambda=0$, 1.0292 at $\lambda=0.02$, and only rises to 1.1253 at $\lambda=0.2$. In fact $\lambda=0.02$ gives the *lowest* test MSE of the three, edging out OLS slightly: a small amount of shrinkage removes some OLS variance in the weakly determined contrast direction essentially for free, while too much shrinkage ($\lambda=0.2$) starts to cost more in bias than it saves in variance. This is exactly why ridge can stabilize the individual coefficients so dramatically without producing a comparably large change in the fitted values: fitted values depend on $X\beta$, and shrinking the coefficient along the direction where X has almost no signal (small $d_j = 0.406$) barely changes $X\beta$ even though it enormously reduces the variance of that otherwise wild coefficient estimate.

Question 3 — What does ridge regression shrink?

Ridge normal equation

Ridge minimizes $L_\lambda(\beta) = (1/2n)\|\tilde{y} - X\beta\|^2 + (\lambda/2)\|\beta\|^2$.

Differentiate and set the gradient to zero: $\nabla L_\lambda(\beta) = -(1/n) X^T(\tilde{y} - X\beta) + \lambda\beta = 0$.

Rearranging, $-(1/n) X^T\tilde{y} + (1/n) X^TX\beta + \lambda\beta = 0$.

Multiplying through by n gives the stated normal equation: $*(X^TX + n\lambda I_p) \beta_\lambda = X^T\tilde{y}.*$

Uniqueness for $\lambda > 0$

X^TX is symmetric positive *semi*-definite (all eigenvalues ≥ 0), regardless of the rank of X — this holds even when X is rank-deficient or $p > n$. Adding $n\lambda I_p$ shifts every eigenvalue up by $n\lambda > 0$, so every eigenvalue of $(X^TX + n\lambda I_p)$ is at least $n\lambda > 0$. A symmetric matrix with strictly positive eigenvalues is positive definite, hence invertible. So the normal equation has the unique solution $\beta_\lambda = (X^TX + n\lambda I_p)^{-1} X^T\tilde{y}$ no matter what the rank of X is — unlike OLS, which requires X^TX itself to be invertible.

Shrinkage in the SVD basis

Let $X = U_r D_r V_r^T$ be the compact SVD, $r = \text{rank}(X)$. Look for a solution of the form $\beta_\lambda = V_r c$ for some $c \in \mathbb{R}^r$ (we'll confirm it's *the* solution since we already showed the solution is unique). Using $X^TX = V_r D_r^2 V_r^T$ and $X^T = V_r D_r U_r^T$, plug into the normal equation:

$$(V_r D_r^2 V_r^T + n\lambda I_p)(V_r c) = V_r D_r^2 c + n\lambda V_r c = V_r (D_r^2 + n\lambda I_r) c.$$

This must equal $X^T\tilde{y} = V_r D_r U_r^T \tilde{y}$. Since V_r has orthonormal columns, matching coefficients gives

$$(D_r^2 + n\lambda I_r) c = D_r U_r^T \tilde{y} \implies c = (D_r^2 + n\lambda I_r)^{-1} D_r U_r^T \tilde{y}.$$

So $\beta_\lambda = V_r (D_r^2 + n\lambda I_r)^{-1} D_r U_r^T \tilde{y}$, and this is the unique solution by the uniqueness argument above. The fitted values are then

$$X\beta_\lambda = U_r D_r V_r^T \beta_\lambda = U_r D_r (D_r^2 + n\lambda I_r)^{-1} D_r U_r^T \tilde{y} = U_r D_r^2 (D_r^2 + n\lambda I_r)^{-1} U_r^T \tilde{y} = \sum_{j=1}^r \frac{d_j^2}{(d_j^2 + n\lambda)} (u_j^T \tilde{y}) u_j.$$

So the fitted-response component along the j -th left singular vector u_j , which at $\lambda = 0$ would simply be $(u_j^T \tilde{y}) u_j$, gets multiplied by

$$\rho_j(\lambda) = d_j^2 / (d_j^2 + n\lambda).$$

Numeric example: $n = 100$, $\lambda = 0.01$, $d = (10, 2, 0.2)$

$n\lambda = 1$, so:

- $\rho_1 = 10^2 / (10^2 + 1) = 100/101 \approx \mathbf{0.9901}$
- $\rho_2 = 2^2 / (2^2 + 1) = 4/5 = \mathbf{0.8000}$
- $\rho_3 = 0.2^2 / (0.2^2 + 1) = 0.04/1.04 \approx \mathbf{0.0385}$

$$df_{\text{eff}}(\lambda) = 1 + \rho_1 + \rho_2 + \rho_3 \approx 1 + 0.9901 + 0.8000 + 0.0385 \approx \mathbf{2.83}.$$

Which direction is shrunk the most? Direction 3 ($d_3 = 0.2$, the smallest singular value) receives the strongest shrinkage — its component is scaled down to under 4% of its unpenalized value, versus 99% and 80% for the other two directions.

Why does shrinking it reduce prediction variance? A small d_j means X has very little “signal” (spread) in that direction, so the corresponding OLS component $(u_j^T \tilde{y}) u_j$ has variance that scales like $1/d_j^2$ — it's estimated almost entirely from noise. Multiplying it by a small ρ_j suppresses that noisy, high-variance component, which is nearly free from a prediction standpoint because that direction barely moves the fitted values (small d_j) anyway.

When could this shrinkage cause substantial bias instead? If the true signal actually has a large coefficient in that low-singular-value direction — i.e., the true regression function genuinely depends strongly on the low-variance direction of X — then shrinking ρ_3 toward 0 removes real signal, not just noise, and the fitted values will be systematically off in that direction, producing meaningful bias. This is the classic ridge trade-off: it helps most when the truth is “smooth” in the sense of concentrating on the high-singular-value directions, and hurts most when the truth genuinely needs the low-singular-value directions.

Question 4 — Ridge regression and optimization

Condition numbers

A_0 has eigenvalues 9 and 0.01, so $\text{cond}(A_0) = 9 / 0.01 = \mathbf{900}$.

$A_\lambda = A_0 + \lambda I$ with $\lambda = 0.09$ shares the same eigenvectors, with eigenvalues shifted up by λ : $9 + 0.09 = 9.09$ and $0.01 + 0.09 = 0.10$. So $\text{cond}(A_\lambda) = 9.09 / 0.10 = \mathbf{90.9}$.

Geometric meaning. Adding λI raises the small eigenvalue proportionally far more than the large one ($0.01 \rightarrow 0.10$ is a $10\times$ increase, versus $9 \rightarrow 9.09$ which is barely a change). The elliptical level sets of the ridge objective are therefore far less elongated than those of the unpenalized objective — the objective is much closer to “round” — which is exactly why ridge makes the optimization landscape better behaved and easier for iterative methods like gradient descent to navigate.

Gradient descent contraction

L_λ is quadratic with gradient $\nabla L_\lambda(\beta) = A_\lambda \beta - (1/n)X^T\tilde{y}$ (A_λ is the Hessian). The ridge minimizer satisfies $A_\lambda \beta_\lambda = (1/n)X^T\tilde{y}$. Subtracting,

$$\beta^{(k+1)} - \beta_\lambda = \beta^{(k)} - \eta(A_\lambda \beta^{(k)} - (1/n)X^T\tilde{y}) - \beta_\lambda = (\beta^{(k)} - \beta_\lambda) - \eta A_\lambda (\beta^{(k)} - \beta_\lambda) = (I - \eta A_\lambda) e^{(k)}.$$

Since $A_\lambda = A_0 + \lambda I$ shares eigenvectors with A_0 , in an eigendirection with unpenalized eigenvalue a (so A_λ -eigenvalue $a + \lambda$), this reduces to a scalar recursion: the component of $e^{(k)}$ along that eigenvector is multiplied by $\mathbf{1 - \eta(a + \lambda)}$ at each step, as required.

Step-size interval that contracts every direction

We need $|1 - \eta(a + \lambda)| < 1$ for both eigenvalues of A_λ , i.e. for $a + \lambda \in \{9.09, 0.10\}$. This holds for all these eigenvalues simultaneously exactly when $0 < \eta < 2/M$, where $M = 9.09$ is the largest eigenvalue of A_λ :

$$\mathbf{0 < \eta < 2/9.09 \approx 0.220}.$$

$\eta = 1/M \approx 0.11$: - Direction with $a + \lambda = 9.09$ (the M -direction): factor = $1 - (1/9.09)(9.09) = \mathbf{0}$. This direction converges in a single step. - Direction with $a + \lambda = 0.10$: factor = $1 - (1/9.09)(0.10) \approx 1 - 0.011 \approx \mathbf{0.989}$. This direction barely moves each iteration.

So the large-eigenvalue direction converges essentially instantly, while the small-eigenvalue (ill-conditioned) direction converges extremely slowly — this asymmetry is a direct readout of the condition number $\text{cond}(A_\lambda) = 90.9$.

$\eta = 2.1/M \approx 0.231$: - Factor for the M -direction: $1 - (2.1/9.09)(9.09) = 1 - 2.1 = \mathbf{-1.1}$, and $|-1.1| > 1$.

Since the initial error has a nonzero component along the M -eigendirection, that component gets multiplied by -1.1 every iteration, growing in magnitude without bound. **Gradient descent diverges** at this step size — it lies outside the contraction interval $(0, 2/M)$ derived above.

Why “fastest GD convergence” is not a valid tuning rule for λ

λ 's job is to control the bias-variance trade-off of the *statistical* estimator — how much we shrink toward zero to trade some bias for less variance and (hopefully) lower prediction error. The condition number of A_λ , and hence how fast gradient descent converges, is a *computational* property of the optimization algorithm we happen to be using to compute β_λ , not a property of how well β_λ predicts new data. Pushing λ larger purely to make gradient descent converge faster will over-shrink the coefficients regardless of whether that over-shrinkage helps or hurts prediction — it optimizes the wrong objective. Moreover, convergence speed is solvable by other means entirely (closed-form solve, better preconditioning, acceleration, Newton's method) without touching λ at all, so there's no reason optimization convenience should be allowed to dictate a quantity that should instead be chosen by validation/CV/GCV against predictive performance.

Question 5 — Selecting the ridge penalty for real-estate prediction

CV curve

$\lambda_{\min} = \mathbf{0.0001}$ (the smallest grid point above 0), with mean CV MSE = **83.01**, $\text{SE}(\lambda_{\min}) = \mathbf{19.15}$.

$\lambda_{1se} = \mathbf{1.6001}$ (largest λ in Λ , including OLS, with $\text{mean CV MSE} \leq \text{mean CV MSE}(\lambda_{\min}) + \text{SE}(\lambda_{\min}) = 83.01 + 19.15 = 102.16$).

Note the SE here is large relative to the gap between candidate λ 's mean MSEs — the CV curve is quite flat/noisy over most of Λ , which is why λ_{1se} ends up so much larger than $\lambda_{\min}$: many λ values, including fairly large ones, fall within one SE of the minimum.

GCV curve

$\lambda_{\text{GCV}} = \mathbf{0.0501}$ (GCV = 83.53, $\text{df}_{\text{eff}} = 6.48$ at that λ).

Final comparison table (refit on all 332 training obs)

Model	λ	train MSE	test MSE	df_{eff}
OLS	0	80.09	66.32	7.00
Ridge ($\lambda_{\min} = 0.0001$)	0.0001	80.09	66.32	6.999
Ridge ($\lambda_{1se} = 1.6001$)	1.6001	100.57	81.39	3.02
Ridge ($\lambda_{\text{GCV}} = 0.0501$)	0.0501	80.30	65.72	6.48

Interpretation. OLS and ridge at $\lambda_{\min}$ are practically indistinguishable here — $\lambda_{\min}$ landed essentially at 0, so almost no shrinkage was selected, meaning CV found little evidence that this dataset's six predictors are collinear or noisy enough to benefit from penalization (unlike the constructed, highly collinear design in Question 2). Ridge at λ_{GCV} applies a modest amount of shrinkage (df_{eff} drops from 7.00 to 6.48) and actually achieves the *lowest* test MSE of the four (65.72), trading a hair of training fit for a slightly better-generalizing fit — a small, believable example of the bias-variance trade-off working in ridge's favor. Ridge at λ_{1se} is a different story: by design it picks a much simpler model ($\text{df}_{\text{eff}} \approx 3.02$, roughly half the effective parameters), and here that extra shrinkage is too aggressive for this particular dataset — both train MSE (100.57) and test MSE (81.39) are clearly worse than the other three. This illustrates that the 1-SE rule prioritizes simplicity/stability over raw predictive accuracy, and that trade-off doesn't always pay off in test MSE for a given dataset, even though it protects against over-fitting the CV curve itself in situations where the curve is noisy (which it clearly is here, given how wide the SE bands are).

Ranking by test MSE alone: λ_{GCV} (65.72) < $\lambda_{\min} \approx$ OLS (66.32) < λ_{1se} (81.39) — but see the caution below about not using this ranking to *choose* the final model.

Why you must not pick the model with the smallest observed test MSE. The four λ values (0, $\lambda_{\min}$, λ_{1se} , λ_{GCV}) were all fixed using only the training data and its folds, *before* looking at the test set — that's what makes the test MSE for each of them a valid, honest estimate of out-of- sample performance. But if you were to now go back and choose whichever of these four happens to have the smallest test MSE, the test set itself would be functioning as an additional round of model selection — turning the “final” evaluation into another tuning step. The reported test MSE for the winner would then be optimistically biased (it was picked *because* it looked best on that particular finite sample of 82 points), and would no longer be a trustworthy estimate of true generalization error. The test set is only valid for *reporting* performance of models chosen in advance, not for *choosing among* them.

explain-to-me Skill

```
---
name: explain-to-me
description: Explains what a STAT 432 homework question is asking, without solving it. Use only when the
student explicitly asks to use explain-to-me on a homework question.
metadata:
  version: "1.0.0"
---
```

Intent

I want help understanding a homework question before I attempt it, not a worked solution. Use this skill only when I explicitly ask you to use it on a specific question.

Instructions

1. Read only the question I named (e.g., “Question 2”), plus any shared setup/preamble it depends on. Ignore other questions in the file.
2. Explain, in your own words:
 - **Goal:** what the question is ultimately asking me to find, show, or compare.
 - **Required work:** what steps/quantities I need to produce (e.g., simulate data, fit a model, report a statistic) — as a checklist, not as code or a derivation.
 - **Relevant lecture material:** point me to the specific Week 3 section(s) that cover the needed concept — “Ridge Regression: Stability Through Shrinkage” or “From a Penalized Objective to a Fitted Ridge Model” — and say briefly why that section applies.
3. If the question has sub-parts (a, b, c, ...), briefly address each one separately.

Rules

- Do not write or outline code, do not derive formulas, and do not state numeric or algebraic answers. If asked to also solve it, decline and remind me this skill is for understanding the prompt only.
- Do not grade, critique, or comment on any work I’ve already done.
- Keep the explanation concise — a short paragraph or checklist per part, not an essay.